

CURV AI — AWS Deployment Checklist

STAGE 0: Audit Complete

- [x] Repository inventory (38 tables, 27 routers, 17 organic adapters, 10 ads adapters)
- [x] Environment variable audit (46+ env vars categorized)
- [x] Database audit (PG 16, RLS, 38 tables, 14 migrations)
- [x] Storage audit (S3, knowledge hub, generated assets)
- [x] Realtime/workers audit (Celery, EventBus, SSE, webhooks)
- [x] Frontend architecture audit (Next.js 15 CSR, API proxy)
- [x] Backend architecture audit (FastAPI, ECS Fargate recommended)
- [x] Security audit (17 findings classified)
- [x] Observability audit (CloudWatch, gaps identified)
- [x] CI/CD audit (broken paths, no OIDC)
- [x] Domain/HTTPS audit (5 different domains in use)
- [x] Cost model (\$25 impossible, \$130 minimum, \$275 small prod)
- [x] Architecture freeze compliance (compliant, no new abstractions)

STAGE 1: AWS Foundation (USER ACTION REQUIRED)

- [] AWS account created and accessible
- [] Administrative IAM user exists
- [] AWS CLI configured locally (`aws configure` or `aws login`)
- [] No long-term access keys created for CI (will use OIDC)
- [] MFA enabled on root and admin accounts
- [] Billing alerts configured (\$25 budget threshold)
- [] ap-south-1 (Mumbai) selected as primary region

STAGE 2: Fix Terraform Blockers (CODE CHANGES)

- [] **CRITICAL:** Add regional ACM certificate for ALB (us-east-1 cert only works for CloudFront)
- [] **CRITICAL:** Fix `ecs_api_memory` default from 1048 → 2048 (invalid Fargate combo)
- [] **CRITICAL:** Use URL-safe DB password (alphanumeric only, no special chars in `random_password`)
- [] **CRITICAL:** Bootstrap S3 state bucket + DynamoDB lock table out-of-band (chicken-and-egg)
- [] **HIGH:** Add `ignore_changes` to `aws_secretsmanager_secret_version.app_secrets` (prevent reset on apply)
- [] **HIGH:** Fix `coalesce(var.ecr*_image, "...")` — empty string is not null
- [] **MEDIUM:** Add Celery beat service (EventBridge Scheduler or singleton Fargate task)
- [] **MEDIUM:** Add SNS email subscription to alerts topic
- [] **MEDIUM:** Fix Redis CloudWatch alarm dimension (per-member cluster ID, not RG ID)
- [] **MEDIUM:** Add ECR lifecycle policy for web repository
- [] **MEDIUM:** Upgrade CloudFront OAI → OAC
- [] **MEDIUM:** Add second NAT gateway for multi-AZ (or accept single-AZ for dev)
- [] **LOW:** Enable S3 public access block on frontend bucket

STAGE 3: Fix CI/CD Pipeline (CODE CHANGES)

- [] **CRITICAL:** Fix deploy.yml Dockerfile paths (`Dockerfile.api` → `apps/api/Dockerfile` , etc.)
- [] **CRITICAL:** Fix deploy.yml build contexts (web → `apps/web-v2` , not `apps/web`)
- [] **HIGH:** Switch from long-lived AWS keys to OIDC + IAM role in GitHub Actions
- [] **MEDIUM:** Add test gate in deploy.yml (don't deploy if tests fail)
- [] **MEDIUM:** Fix security.yml — remove `|| true` from audit steps, fix `requirements.txt` reference
- [] **MEDIUM:** Add `pnpm-lock.yaml` to web-v2 and use `--frozen-lockfile` in Dockerfile
- [] **LOW:** Configure Terraform remote state backend in CI

STAGE 4: Fix Application Issues (CODE CHANGES)

- [] **HIGH:** Fix OAuth `state` parameter — use signed JWT with nonce, not bare `brand_id`
- [] **HIGH:** Fix X/Twitter PKCE — persist `code_verifier` for `exchange_code`
- [] **HIGH:** Unify domain — pick ONE domain (curv.app), update CORS, `WEB_URL`, `EMAIL_FROM`, OAuth redirect URLs, Terraform vars, `render.yaml`
- [] **MEDIUM:** Make CORS origins env-driven (`CORS_ORIGINS` setting)
- [] **MEDIUM:** Add production safety check — reject `change-me-*` secrets when `APP_ENV != local`
- [] **MEDIUM:** Add non-root USER to api/worker/web-v2 Dockerfiles
- [] **MEDIUM:** Move video generation to Celery task (currently blocks HTTP request ≤600s)
- [] **MEDIUM:** Move knowledge upload processing to Celery task
- [] **MEDIUM:** Add file size limit to `/knowledge/upload` endpoint
- [] **MEDIUM:** Add presigned URL download endpoint for reports/assets
- [] **MEDIUM:** Store uploaded knowledge docs to S3 (currently metadata-only)
- [] **MEDIUM:** Fix admin "ops" endpoints — add superadmin role distinction
- [] **MEDIUM:** Fix API tokens — store hashed in DB, wire to auth
- [] **MEDIUM:** Add structured JSON logging for production
- [] **MEDIUM:** Add error tracking (Sentry or CloudWatch RUM)
- [] **LOW:** Rotate refresh tokens on use
- [] **LOW:** Gate `/metrics` and `/docs` in production
- [] **LOW:** Fix `.env.example` duplicate Stripe/Razorpay blocks
- [] **LOW:** Add missing env vars to `.env.example` (`CELERY_`, `DB_POOL_`, etc.)

STAGE 5: Fix Database Issues (MIGRATION CHANGES)

- [] **MEDIUM:** Add RLS policies to 11 unprotected tables (`council_`, `knowledge_`, `review_*`, `campaign_performance`)
- [] **MEDIUM:** Add `tenant_id` column to `campaign_performance` table
- [] **LOW:** Create monthly partition automation for `metric_events` (pg_partman or scheduled job)
- [] **LOW:** Switch SSL to `verify-full` with RDS CA bundle
- [] **LOW:** Create dedicated app DB role (non-owner) for RLS to work

STAGE 6: Infrastructure Deployment

- [] Bootstrap S3 state bucket + DynamoDB lock table manually
- [] Run `terraform init` in `infra/terraform/`
- [] Create `terraform.tfvars` with curv.app domain, t3.small instances

- ☐ Run `terraform plan` and review
- ☐ Run `terraform apply` for dev/staging environment
- ☐ Verify all resources created successfully
- ☐ Populate Secrets Manager with real values (out-of-band, not via Terraform)
- ☐ Create dedicated DB app role and grants
- ☐ Run Alembic migrations via one-off ECS task

STAGE 7: Application Deployment

- ☐ Build Docker images for api, web-v2, worker
- ☐ Push images to ECR
- ☐ Deploy API service to ECS Fargate
- ☐ Deploy worker service to ECS Fargate
- ☐ Deploy beat scheduler (EventBridge or singleton task)
- ☐ Deploy web-v2 to ECS Fargate (NOT S3 — requires Node server)
- ☐ Verify `/health` and `/health/ready` endpoints
- ☐ Verify ALB HTTPS listener works
- ☐ Verify CloudFront distribution serves frontend
- ☐ Run smoke tests against staging

STAGE 8: DNS Cutover

- ☐ Update Route53 records to point to AWS (api.curv.app → ALB, app.curv.app → CloudFront)
- ☐ Update OAuth redirect URLs in all provider consoles (Google, Meta, LinkedIn, X, etc.)
- ☐ Update `WEB_URL` in `.env` to `https://app.curv.app`
- ☐ Update CORS origins to include `https://app.curv.app`
- ☐ Update `NEXT_PUBLIC_API_BASE` build arg to `https://api.curv.app`
- ☐ Verify DNS propagation
- ☐ Verify OAuth flows work end-to-end
- ☐ Verify webhook endpoints receive requests

STAGE 9: Monitoring & Rollback

- ☐ Add SNS email subscription for CloudWatch alarms
- ☐ Configure CloudWatch dashboard
- ☐ Set up error tracking (Sentry or equivalent)
- ☐ Verify Celery worker health
- ☐ Verify Celery beat schedule running
- ☐ Test rollback procedure (revert to previous ECR image tag)
- ☐ Document rollback runbook
- ☐ Set up backup verification (RDS automated snapshots)

STAGE 10: E2E Testing

- ☐ Create proper `playwright.config.ts` for web-v2
- ☐ Convert `e2e-audit.mjs` to Playwright test specs

- [] Add E2E tests to CI pipeline
- [] Test: authentication (login, register, social)
- [] Test: onboarding flow
- [] Test: dashboard + Orb chat
- [] Test: campaign creation
- [] Test: creative studio
- [] Test: knowledge hub upload
- [] Test: integrations/connections
- [] Test: OAuth callback flow
- [] Test: responsive mobile UI

AWS Permissions Required for Next Deployment Phase

Devin would need the following IAM permissions (DO NOT create yet):

```
# Terraform managed permissions
iam:CreateRole, iam:PutRolePolicy, iam:AttachRolePolicy, iam:CreatePolicy, iam:GetRole, iam:ListRoles
ec2:CreateVpc, ec2:CreateSubnet, ec2:CreateSecurityGroup, ec2:CreateInternetGateway, ec2:CreateNatGateway
ecs:CreateCluster, ecs:RegisterTaskDefinition, ecs:CreateService, ecs:UpdateService
rds:CreateDBInstance, rds:CreateDBSubnetGroup, rds:ModifyDBInstance
elasticache:CreateReplicationGroup, elasticache:CreateCacheSubnetGroup
s3:CreateBucket, s3:PutBucketPolicy, s3:PutBucketVersioning, s3:PutEncryptionConfiguration
cloudfront:CreateDistribution, cloudfront:UpdateDistribution
route53:CreateHostedZone, route53:ChangeResourceRecordSets
acm:RequestCertificate, acm:DescribeCertificate
elasticloadbalancing:CreateLoadBalancer, elasticloadbalancing:CreateListener, elasticloadbalancing:CreateTargetGroup
secretsmanager:CreateSecret, secretsmanager:PutSecretValue, secretsmanager:DescribeSecret
ecr:CreateRepository, ecr:PutLifecyclePolicy
logs:CreateLogGroup, logs:PutRetentionPolicy
sns:CreateTopic, sns:Subscribe
wafv2:CreateWebACL, wafv2:AssociateWebACL
kms:CreateKey, kms:CreateAlias
```

Recommended approach: Create a dedicated `terraform-deployer` IAM role with these permissions, assume it via OIDC from GitHub Actions. Do NOT create long-lived access keys.